

# TalkToTasks: Technical Description

TalkToTasks turns an English meeting recording into a raw transcript, a domain-corrected transcript, and a structured meeting record (summary, minutes, decisions, action items)[cite: 1]. It runs as one automated pipeline behind a Streamlit interface[cite: 1]. Nothing is hardcoded: every output is generated from the uploaded audio[cite: 1].

## 1 Models and Roles

| Stage | Role                            | Model / Where it runs                                                                                                                                                              |
|-------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Speech-to-text                  | AssemblyAI (default, with speaker labels and optional glossary word-boost). Fallback: faster-whisper small (int8, CPU, VAD filter, English)<br><i>AssemblyAI cloud API / local</i> |
| 2     | LLM 1: transcript refiner       | openai/gpt-oss-120b (temperature 0, low reasoning effort)<br><i>Groq API</i>                                                                                                       |
| 3     | LLM 2: minutes & task extractor | openai/gpt-oss-120b (temperature 0, medium reasoning effort, JSON mode)<br><i>Groq API</i>                                                                                         |
| 4     | Verifier (rule-based, no model) | Deterministic Python checks<br><i>Local</i>                                                                                                                                        |

Stages 2 and 3 are fully separate: each has its own prompt file (`prompts/refine.txt`, `prompts/minutes.txt`), its own API call, its own output format and its own model setting (`REFINER.MODEL`, `MINUTES.MODEL`), so either can be swapped independently[cite: 1]. The model for each stage can also be changed in the UI we have provided the option for it though[cite: 1].

## 2 Data Flow Between Stages

Audio upload

```
| validate (exists, non-empty, supported type) -> ffmpeg -> 16 kHz mono WAV
v
[1] Speech-to-text -> segments {start, end, speaker, text}
v
[2] LLM #1 Refiner -> same segments, corrected text (raw_text kept)
|   long segments split (<=60 words), batches (<=20 segments / 500 words)
|   I/O as numbered lines "[n] text"; unsafe edits revert to raw line
v
[3] LLM #2 Minutes -> JSON {summary, minutes, decisions, action_items}
|   input: refined transcript only, as "[mm:ss] Speaker: text"
```

```

|   output validated against a Pydantic schema, one retry if invalid
v
[4] Verifier -> checks evidence, numbers, owners, deadlines
v
Outputs: raw transcript, refined transcript, record (.md, .json), zip

```

The orchestration lives in `pipeline/run.py` (`process_audio`), which runs the stages strictly in order and reports status to the UI at each step[cite: 1]. The raw and refined transcripts are both retained per segment, which powers the side-by-side comparison[cite: 1].

### 3 How the Requirements are Met

**Refinement without changing meaning.** The refiner prompt only allows fixes to technical terms, acronyms and product names (for example “see eye pipeline” becomes “CI pipeline”), and requires the same line IDs in the same order[cite: 1]. Code then enforces this independently of the model[cite: 1]:

- Any edit that changes the count of numbers or negations in a line is rejected (signature check)[cite: 1].
- Any edit touching a number, number word, negation, or a capitalised mid-sentence word (likely a name) is rejected[cite: 1].
- Any line rewritten so heavily that its similarity to the original falls below 0.6 is rejected[cite: 1].

If the model returns mismatched IDs or the API is unavailable for a batch, the original text is kept and the reason is logged as a flag, which the UI shows[cite: 1].

**Not inventing owners or deadlines.** The minutes prompt defines a decision as requiring explicit agreement (proposals such as “maybe we should” are excluded), defines an action item as a clear commitment, and requires owner and deadline to be “unspecified” unless stated[cite: 1]. Code reinforces this: `normalize` converts empty or “none/n/a” values to unspecified, and the verifier resets any owner or deadline whose words do not appear in the transcript (including speaker labels like “A” or “B”)[cite: 1].

**Grounding.** Every decision and action item carries an exact evidence quote and a timestamp[cite: 1]. The verifier checks that the quote exists in the transcript (exact or at least 0.8 fuzzy match) and that no number appears that is absent from the transcript[cite: 1]. Each item is marked “verified” or “check evidence” in the UI[cite: 1].

**Error handling.** Missing files, empty files, unsupported extensions, unreadable or silent audio, and audio with no detectable speech raise a clear `AudioError`, shown in the interface[cite: 1]. Missing API keys, rate limits and a missing `ffmpeg` also produce readable messages[cite: 1]. LLM calls retry with backoff on transient errors[cite: 1]. If AssemblyAI fails in “auto” mode, the pipeline falls back to local Whisper[cite: 1].

### 4 Interface and Outputs

The Streamlit app (`app.py`) lets the user upload audio, optionally enter a domain glossary, choose the speech and language models, and start processing[cite: 1]. It shows live step status (Speech-to-text, Refine, Minutes), then tabs for Overview, Transcripts (raw and refined side by side, with word-level diff highlighting, search, speaker filter and “changed lines only”), Minutes, Decisions & Actions (with evidence), and Downloads[cite: 1]. Previous runs can be reopened from the sidebar[cite: 1].

| Output             | Format | Contents                                                                       |
|--------------------|--------|--------------------------------------------------------------------------------|
| Raw transcript     | .txt   | Timestamped, speaker-labelled speech-to-text output                            |
| Refined transcript | .txt   | Same lines after domain-term correction                                        |
| Meeting record     | .md    | Summary, minutes, decisions, action items, verification notes (human-readable) |
| Meeting record     | .json  | Same content with evidence, timestamps and verified flags (machine-readable)   |
| All of the above   | .zip   | One-click download                                                             |

The Markdown and JSON are produced from the same record object, so they always contain the same decisions and tasks[cite: 1]. Missing owners and deadlines appear as “unspecified” in both[cite: 1].

## 5 Sample Run

`samples/team_meeting.wav` produces the outputs in `outputs/team_meeting/`[cite: 1]. The record contains one decision (keep the current database schema; the Redis caching idea is correctly left as not decided) and three action items: deploy the CI pipeline (Rahul, Friday), update the documentation (owner and deadline unspecified), and share benchmark results (Priya, Monday)[cite: 1]. The vague remark about thinking about load testing is correctly not listed as a task[cite: 1]. The end-to-end run took about 21 seconds[cite: 1].

## 6 Known Limitations

- The number-protection rule means the refiner will not change a term that contains a digit, so a mis-heard “P95” transcribed as “page 95” is left as is (visible in the sample transcript)[cite: 1].
- Speaker labels are generic (“A”, “B”), so owners are only recorded when a name is spoken[cite: 1].
- The default speech path needs an AssemblyAI key and internet access; the local Whisper fallback has no speaker labels[cite: 1].
- Both language-model stages currently use the same model; they are separate stages and can be pointed at different models through configuration[cite: 1].

## 7 Repository Layout and Setup

|                              |                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app.py</code>          | Streamlit interface                                                                                                                            |
| <code>.streamlit/</code>     | <code>config.toml</code> (theme)                                                                                                               |
| <code>pipeline/</code>       | <code>stt.py</code> , <code>refine.py</code> , <code>minutes.py</code> , <code>verify.py</code> , <code>export.py</code> , <code>run.py</code> |
| <code>prompts/</code>        | <code>refine.txt</code> , <code>minutes.txt</code>                                                                                             |
| <code>outputs/</code>        | sample generated outputs                                                                                                                       |
| <code>samples/</code>        | shareable test recordings                                                                                                                      |
| <code>run_pipeline.py</code> | command-line entry point                                                                                                                       |

**Setup:** run `pip install -r requirements.txt`, install `ffmpeg`, create a `.env` file with `GROQ_API_KEY` and `ASSEMBLYAI_API_KEY`, then run `streamlit run app.py` from the project root[cite: 1].